

Agentic AI Minimum Guide

송경모, 한국공학대학교

2026-04-08

Windows 환경에서 OpenAI API를 이용한 최소 수준의 Agentic AI 과제 수행 가이드

이 문서는 **Windows GUI 사용자**인 대학 교양 수강생이, **OpenAI API key**를 발급받아 **Python + Streamlit**으로 자신의 과업을 수행하는 최소 수준의 agentic AI 프로그램을 실행해보도록 안내하기 위한 실습용 가이드이다.

구성은 다음 5부분으로 연결된다.

1. 최소 수준의 .py 프레임 제공
2. Python 설치법
3. 메모장과 VS Code 같은 에디터 기본 사용법
4. OpenAI API key 발급 및 .env 저장 관리법
5. Windows Terminal에서 .venv 가상환경 활성화 후 .py 실행법

1. 최소 수준의 .py 프레임

1-1. 이 프레임의 목적

이 프레임은 다음을 한 번에 만족하도록 설계하였다.

- 웹 브라우저에서 실행되는 **Streamlit 챗봇 화면** 제공
- OpenAI API key를 **.env 파일**에 보관하고 불러오기
- system, assistant, user 메시지 역할을 **명시적으로 구분**

- **multi-turn 대화** 유지
- **few-shot prompting** 가능
- 학생이 자신의 **로컬 문서(txt, md, pdf)**와 **참고 URL**을 지정 가능
- 각 부분에 교육용 주석 포함
- 이후 학생이 ChatGPT에 이 파일을 보여주며
 - "이 구조를 유지한 채 내 과업 어쩌구 저쩌구를 수행하는 agent로 바꿔줘"라고 요청할 수 있도록 설계

1-2. 왜 이 프레임이 '최소 수준'인가

OpenAI는 현재 **Responses API**를 새 프로젝트에 권장하고 있으며, Chat Completions도 여전히 지원한다. 다만 교육 초반에는 system / assistant / user 메시지 구조를 학생이 직접 이해하는 것이 중요하므로, 본 실습 프레임은 **메시지 구조가 가장 직관적인 Chat Completions 방식**을 사용한다. OpenAI 공식 문서에 따르면 Responses API는 새 프로젝트에 권장되지만 Chat Completions도 계속 지원된다.

또한 OpenAI 공식 문서는 API key를 **환경변수로 안전하게 관리**하라고 안내하며, API key를 클라이언트 코드나 공개 코드에 노출하지 말라고 명시한다. 본 가이드가 .env를 사용하는 이유가 여기에 있다.

1-3. 권장 프로젝트 폴더 구조

과제를 하는 학생에게는 다음 정도의 구조가 가장 단순하고 관리하기 쉽다.

```

AI_Agent_Project/
├── app.py           ← Streamlit 실행 파일
├── .env             ← OPENAI_API_KEY 저장
├── requirements.txt ← 필요한 라이브러리 목록
├── README.md        ← 실행 메모(선택)
├── data/            ← 참고 문서 저장 폴더
│   ├── sample1.txt
│   ├── sample2.md
│   └── sample3.pdf
└── .venv/           ← 가상환경 폴더(자동 생성)
  
```

1-4. 먼저 저장할 requirements.txt

아래 내용을 requirements.txt라는 이름으로 저장한다.

```
openai
streamlit
python-dotenv
pypdf
requests
beautifulsoup4
```

1-5. 먼저 저장할 .env

아래처럼 .env 파일을 만든다. 실제 키 값은 학생 본인의 키로 바꾼다.

OPENAI_API_KEY=여기에_본인의_API_KEY를_붙여넣기

1-6. 최소 수준의 app.py 전체 예시

아래 코드를 그대로 복사하여 app.py로 저장하면 된다.

```
# =====
# app.py
# -----
# 목적:
# - OpenAI API를 이용한 최소 수준의 Streamlit 챗봇 예제
# - system / assistant / user 메시지 구조 학습
# - multi-turn 대화 유지
# - few-shot prompting 실험 가능
# - 로컬 참고문서(txt, md, pdf)와 참고 URL 내용을 함께 전달 가능
#
# 실행 전 준비:
# 1) .env 파일에 OPENAI_API_KEY 저장
# 2) 필요한 패키지 설치
# 3) streamlit run app.py 실행
# =====

import os
from io import BytesIO
```

```

import requests
import streamlit as st
from bs4 import BeautifulSoup
from dotenv import load_dotenv
from openai import OpenAI
from pypdf import PdfReader

# -----
# 1. 환경변수(.env) 로드
# -----
# .env 파일에 저장된 OPENAI_API_KEY를 읽어온다.
# 키를 코드 안에 직접 쓰지 않고 별도 파일에 두는 이유는
# 보안과 관리 편의성 때문이다.
load_dotenv()
api_key = os.getenv("OPENAI_API_KEY")

if not api_key:
    raise ValueError(
        "OPENAI_API_KEY를 찾지 못했습니다. .env 파일에 API key를 저장했는지 확인하세요."
    )

# -----
# 2. OpenAI 클라이언트 생성
# -----
client = OpenAI(api_key=api_key)

# -----
# 3. 참고문서 읽기 함수
# -----
# 업로드된 파일의 내용을 텍스트로 바꾼다.
# 최소 예제로 txt, md, pdf만 지원한다.
# 필요하다면 이후 docx, xlsx 등으로 확장 가능하다.
def read_uploaded_file(uploaded_file) -> str:
    file_name = uploaded_file.name.lower()

```

```

# txt / md 파일 처리
if file_name.endswith(".txt") or file_name.endswith(".md"):
    return uploaded_file.read().decode("utf-8", errors="ignore")

# pdf 파일 처리
elif file_name.endswith(".pdf"):
    pdf_bytes = BytesIO(uploaded_file.read())
    reader = PdfReader(pdf_bytes)
    pages = []
    for page in reader.pages:
        pages.append(page.extract_text() or "")
    return "\n".join(pages)

# 지원하지 않는 파일 형식
else:
    return f"[지원하지 않는 파일 형식: {uploaded_file.name}]"

# -----
# 4. 참고 URL 읽기 함수
# -----
# URL의 본문 텍스트를 매우 단순하게 추출한다.
# 웹페이지 구조가 제각각이므로 완벽하지는 않다.
# 그래도 최소 수준의 참고자료 결합 예제로는 충분하다.
def read_url_text(url: str) -> str:
    try:
        headers = {
            "User-Agent": "Mozilla/5.0"
        }
        response = requests.get(url, headers=headers, timeout=10)
        response.raise_for_status()

        soup = BeautifulSoup(response.text, "html.parser")

        # script, style 제거
        for tag in soup(["script", "style", "noscript"]):
            tag.decompose()

```

```

text = soup.get_text(separator="\n")
lines = [line.strip() for line in text.splitlines() if line.strip()]
clean_text = "\n".join(lines)

```

```

# 너무 긴 경우 앞부분만 사용
return clean_text[:6000]

```

```

except Exception as e:
    return f"[URL 읽기 실패: {url} / 오류: {e}]"

```

```

# -----
# 5. 메시지 구성 함수
# -----
# system / assistant / user 역할을 분명히 구분해서 메시지 리스트를 만든다.
# few-shot prompting을 위해 assistant 예시도 미리 넣어둘 수 있다.
def build_messages(
    system_prompt: str,
    few_shot_user: str,
    few_shot_assistant: str,
    reference_text: str,
    conversation_history: list,
    current_user_input: str,
):
    messages = []

    # (1) system 메시지: 챗봇의 역할, 목적, 원칙 정의
    messages.append({
        "role": "system",
        "content": system_prompt
    })

    # (2) few-shot 예시: 모델이 어떤 식으로 답해야 하는지 예시 제공
    # 비워두면 자동으로 생략된다.
    if few_shot_user.strip() and few_shot_assistant.strip():
        messages.append({
            "role": "user",

```

```

        "content": few_shot_user
    })
    messages.append({
        "role": "assistant",
        "content": few_shot_assistant
    })

# (3) 참고자료가 있으면 user 메시지로 함께 넣는다.
# 아주 정교한 RAG는 아니지만, 최소 수준의 로컬 문서 결합 방식이다.
if reference_text.strip():
    messages.append({
        "role": "user",
        "content": (
            "다음은 참고자료이다. 필요할 때만 활용하고, 참고자료에 없는 내용은 임의로 단정하지 마세요."
            f"[참고자료 시작]\n{reference_text}\n[참고자료 끝]"
        )
    })

# (4) 이전 대화 기록 추가: multi-turn 대화를 가능하게 함
messages.extend(conversation_history)

# (5) 이번 턴의 사용자 입력 추가
messages.append({
    "role": "user",
    "content": current_user_input
})

return messages

# -----
# 6. Streamlit 기본 화면 설정
# -----
st.set_page_config(page_title="My Agentic AI Starter", layout="wide")
st.title("My Agentic AI Starter")
st.caption("OpenAI API + Streamlit + .env + 참고문서/URL + Multi-turn 예제")

```

```

# -----
# 7. 세션 상태(session_state) 초기화
# -----
# Streamlit은 새로고침마다 코드가 다시 실행되므로,
# 대화 기록은 session_state에 저장해야 multi-turn이 유지된다.
if "chat_history" not in st.session_state:
    st.session_state.chat_history = []

# -----
# 8. 사이드바: 프롬프트 설정 영역
# -----
with st.sidebar:
    st.header("설정")

    # system prompt
    system_prompt = st.text_area(
        "System Prompt",
        value=(
            "너는 사용자의 과업을 정확하고 구조적으로 돕는 AI assistant다. "
            "모르면 모른다고 말하고, 근거가 약하면 추정이라고 밝혀라. "
            "답변은 핵심부터 정리하고, 필요한 경우 단계별로 설명하라."
        ),
        height=180,
    )

    # few-shot 예시 입력
    st.subheader("Few-shot 예시(선택)")
    few_shot_user = st.text_area(
        "예시 user 메시지",
        value="예: 어떤 기업의 사업모델을 3단 구조로 요약해줘.",
        height=80,
    )
    few_shot_assistant = st.text_area(
        "예시 assistant 메시지",
        value="예: 1) 고객 2) 가치제안 3) 수익구조의 3단 구조로 요약하겠습니다.",
        height=100,
    )

```



```

# 사용할 모델명
model_name = st.text_input("모델 이름", value="gpt-5")

st.markdown("---")
st.subheader("참고문서 업로드")
uploaded_files = st.file_uploader(
    "txt / md / pdf 파일 업로드",
    type=["txt", "md", "pdf"],
    accept_multiple_files=True,
)

st.subheader("참고 URL 입력")
url_text = st.text_area(
    "URL을 한 줄에 하나씩 입력",
    value="",
    height=120,
    placeholder="https://example.com/article1\nhttps://example.com/article2",
)

clear_button = st.button("대화 기록 초기화")
if clear_button:
    st.session_state.chat_history = []
    st.success("대화 기록을 지웠습니다.")

# -----
# 9. 참고자료 텍스트 만들기
# -----
reference_parts = []

# 로컬 파일 내용 결합
if uploaded_files:
    for file in uploaded_files:
        file_text = read_uploaded_file(file)
        reference_parts.append(f"\n로컬 파일: {file.name}\n{file_text[:6000]}")

# URL 내용 결합

```

```

if url_text.strip():
    urls = [u.strip() for u in url_text.splitlines() if u.strip()]
    for url in urls:
        url_content = read_url_text(url)
        reference_parts.append(f"\n[참고 URL: {url}]\n{url_content}")

```

```

reference_text = "\n\n".join(reference_parts)

```

```

# -----
# 10. 이전 대화 출력
# -----

```

```

for msg in st.session_state.chat_history:
    with st.chat_message(msg["role"]):
        st.markdown(msg["content"])

```

```

# -----
# 11. 현재 사용자 입력 받기
# -----

```

```

user_input = st.chat_input("질문이나 과업 지시를 입력하세요.")

```

```

if user_input:
    # 사용자 메시지 화면 출력
    with st.chat_message("user"):
        st.markdown(user_input)

    # OpenAI에 전달할 전체 메시지 구성
    messages = build_messages(
        system_prompt=system_prompt,
        few_shot_user=few_shot_user,
        few_shot_assistant=few_shot_assistant,
        reference_text=reference_text,
        conversation_history=st.session_state.chat_history,
        current_user_input=user_input,
    )

```

```

try:

```

```

# -----
# 12. OpenAI API 호출
# -----
# 교육 목적상 messages 구조가 분명한 chat.completions.create 사용
response = client.chat.completions.create(
    model=model_name,
    messages=messages,
    temperature=0.3,
)

assistant_reply = response.choices[0].message.content

except Exception as e:
    assistant_reply = f"오류가 발생했습니다: {e}"

# assistant 응답 화면 출력
with st.chat_message("assistant"):
    st.markdown(assistant_reply)

# 실제 multi-turn 유지를 위해 이번 턴을 기록에 저장
st.session_state.chat_history.append({
    "role": "user",
    "content": user_input
})
st.session_state.chat_history.append({
    "role": "assistant",
    "content": assistant_reply
})

```

1-7. 학생이 이 프레임에서 직접 수정해야 할 핵심 위치

학생은 Python 문법을 몰라도, 아래 부분만 바꾸면 기본 실험은 가능하다.

A. 챗봇의 성격과 원칙 설정 system_prompt 기본값 부분을 바꾼다.

```

system_prompt = st.text_area(
    "System Prompt",
    value=(

```

"너는 사용자의 과업을 정확하고 구조적으로 돕는 AI assistant다. "
"모르면 모른다고 말하고, 근거가 약하면 추정이라고 밝혀라. "
"답변은 핵심부터 정리하고, 필요한 경우 단계별로 설명하라."

```
),  
height=180,  
)
```

예:

- "너는 대학생의 발표자료 구성을 도와주는 assistant다."
- "너는 지역 문화유산 답사 계획을 짜주는 assistant다."
- "너는 중소기업 마케팅 아이디어를 구조화해주는 assistant다."

B. few-shot 예시 바꾸기 아래 두 칸은 "사용자가 이렇게 물으면, assistant는 이런 형식으로 답하라"는 예시이다.

```
few_shot_user = st.text_area(...)  
few_shot_assistant = st.text_area(...)
```

예:

- user 예시: "한 기업의 사업모델을 표로 정리해줘."
- assistant 예시: "다음 4열 표로 정리하겠습니다: 고객, 문제, 해결책, 수익원."

C. 모델 이름 바꾸기

```
model_name = st.text_input("모델 이름", value="gpt-5")
```

강의 시점의 사용 가능 모델명으로 바뀌서 쓴다.

D. 실제 과업 입력 웹 화면 하단 입력창에 학생이 자기 과업을 입력한다.

예:

- "내가 업로드한 PDF를 바탕으로 5분 발표용 요약을 만들어줘."
- "첨부 자료를 보고 핵심 개념 7개를 뽑아줘."
- "참고 URL 내용을 비교해서 장단점을 표로 정리해줘."

1-8. 이 프레임을 바탕으로 학생이 자신의 agent를 확장하는 방식

학생은 이 app.py를 ChatGPT 대화창에 붙여 넣고 다음처럼 요청하면 된다.

이 코드를 유지하되, 나의 과업은 '중소기업의 신제품 아이디어를 평가하고 점수화하는 agent' 다음을 추가해줘.

- 1) 평가항목 5개 점수화
- 2) 총점 계산
- 3) 결과를 표로 보여주기
- 4) 필요하면 외부 function calling 구조도 넣기
- 5) 각 함수에 교육용 주식 달기

또는

이 코드를 유지하되, 나의 과업은 '학생 보고서를 평가 루브릭에 따라 채점하는 agent'이다.

- 업로드된 파일 내용 요약
 - 평가 기준별 점수
 - 총평 생성
 - 결과 다운로드 기능
- 을 추가해줘.

1-9. function calling이 필요한 경우 학생이 이해해야 할 최소 개념

agentic AI라고 할 때 핵심은 "모델이 그냥 말만 하는 것"이 아니라, 필요하면 **외부 함수(tool)** 를 호출해 실제 작업을 수행하는 구조까지 포함한다는 점이다. OpenAI는 Responses API가 이런 agentic 작업과 도구 호출에 더 적합하다고 안내한다.

예를 들어 학생의 과업이 다음과 같다면 함수 호출이 필요할 수 있다.

- 날짜 계산하기
- CSV 파일 읽고 통계 구하기
- 인터넷에서 특정 형식의 데이터 수집하기
- 학교 공지 PDF 여러 개를 합쳐서 정리하기
- 사용자 입력값을 점수화하는 계산 수행하기

이때 ChatGPT에게 다음처럼 요청하면 된다.

이 app.py를 유지하되,

'강의계획서 일정표를 분석해서 시험일과 과제일을 표로 뽑아주는 agent'로 확장해줘.

필요한 function calling 함수를 포함하고,
각 함수 위에 '이 함수가 무엇을 하는지'를 설명하는 주석을 달아줘.

2. Python 설치법 안내

2-1. Windows에서 별도 Linux/WSL 없이 가능한가

가능하다. 학생 과제 수준의 OpenAI API + Streamlit 앱은 **Windows용 Python만으로 충분히 수행 가능하다**. 별도 Linux나 WSL은 필수 조건이 아니다.

2-2. 어디서 설치할 것인가

Python은 공식 사이트 **python.org**에서 설치하는 것이 가장 안전하다. Python.org는 현재 최신 Python 3 다운로드를 제공하며, Windows용 설치 관리자도 공식 제공한다. 2026년 4월 현재 Python.org 다운로드 페이지는 Python 3.14.3을 제시하고 있다.

2-3. 설치 절차

1. 웹 브라우저에서 Python 공식 다운로드 페이지에 들어간다.
2. **Windows용 설치 파일**을 내려받는다.
3. 설치 프로그램 실행 시 다음을 확인한다.
 - Add python.exe to PATH 체크
 - 가능하면 기본 설치 또는 표준 설치 선택
4. 설치 완료 후 Windows Terminal 또는 명령 프롬프트를 연다.
5. 아래 명령으로 설치 확인:

```
python --version
```

또는

```
py --version
```

정상 설치되면 Python 버전이 보인다.

2-4. 수업용 권장 버전 기준

수업에서는 굳이 가장 최신 기능을 쫓지 말고, **공식 최신 안정 버전** 또는 학교 PC 환경에서 잘 동작하는 최신 3.x 버전을 쓰면 된다. venv는 Python 표준 라이브러리에 포함되어 있으며, 별도 설치 없이 사용할 수 있다.

3. 메모장과 VS Code 등 에디터 기본 사용법 안내

3-1. 최소 수준: 메모장으로도 가능한가

가능하다. 정말 최소 수준이라면 학생은 다음만 할 수 있어도 된다.

- 새 텍스트 파일 열기
- ChatGPT가 생성해준 코드 붙여넣기
- 파일명을 app.py로 저장하기
- 같은 방식으로 .env, requirements.txt 저장하기

단, Windows 메모장은 파일명을 잘못 저장하면 app.py.txt가 될 수 있다. 따라서 저장 시 아래를 지켜야 한다.

- 파일 형식: **모든 파일**
- 파일명: app.py
- 인코딩: **UTF-8** 권장

3-2. 권장 수준: VS Code 사용

학생에게는 가능하면 **VS Code**를 권장하는 편이 낫다. 이유는 다음과 같다.

- .py, .env, .txt 파일을 구분해서 보기 쉽다.
- 들여쓰기와 괄호를 보기 쉽다.
- 저장 실수가 적다.
- 오류 위치를 찾기 쉽다.

3-3. VS Code에서 필요한 최소 동작

학생이 VS Code에서 할 줄 알아야 할 최소 동작은 다음 정도면 충분하다.

1. 폴더 열기

- AI_Agent_Project 폴더를 연다.

2. 새 파일 만들기
 - app.py
 - .env
 - requirements.txt
3. 코드 붙여넣기 후 저장
 - Ctrl + V
 - Ctrl + S
4. 파일명 확인
 - 확장자가 정확한지 확인
5. 터미널 열기
 - 메뉴 또는 Ctrl + ~

3-4. 학생에게 강조할 저장 규칙

- Python 코드는 반드시 .py
 - API key 저장 파일은 반드시 .env
 - 라이브러리 목록은 requirements.txt
 - 프로젝트 폴더 안에서 파일이 흩어지지 않게 관리
-

4. OpenAI API key 발급 및 .env 저장 관리법

4-1. API key는 ChatGPT 구독과 별개라는 점

학생이 흔히 혼동하는 점이 있다. **ChatGPT 웹 서비스 이용**과 **OpenAI API 사용**은 별개이다. API는 OpenAI Platform에서 key를 발급받고, 해당 key를 이용한 호출량에 따라 과금될 수 있다. OpenAI 공식 개발자 문서는 API key를 생성하여 API 호출에 사용하라고 안내한다.

4-2. API key 발급 절차

OpenAI 도움말과 개발자 문서에 따르면, API key는 OpenAI Developer Platform의 API Keys 페이지에서 **Create new secret key**를 눌러 생성한다. 권한은 생성 시점 또는 이후 편집 화면에서 설정할 수 있다.

학생에게는 다음 순서로 안내하면 된다.

1. OpenAI Platform 계정으로 로그인

2. API Keys 페이지로 이동
3. **Create new secret key** 클릭
4. key 이름을 적절히 지정
5. 생성된 key를 즉시 복사
6. 그 값을 .env 파일에 저장

예:

OPENAI_API_KEY=sk-xxxxxxxxxxxxxxxxxxxxxxxxxxxx

4-3. key 관리 원칙

API key는 **비밀번호와 같은 비밀정보**다. OpenAI 공식 문서도 key를 다른 사람과 공유하거나 브라우저/앱 같은 클라이언트 코드에 노출하지 말라고 명시한다.

학생에게는 다음을 분명히 말해야 한다.

절대 하면 안 되는 것

- app.py 안에 key를 직접 입력
- 과제 제출 파일 안에 key 포함
- 카카오톡, 이메일, 단체 채팅방에 key 공유
- GitHub 공개 저장소에 업로드
- 발표 화면 캡처에 key가 보이도록 방치

해야 하는 것

- .env 파일에만 저장
- 타인에게 보여주지 않기
- 의심되면 즉시 폐기하고 새 key 발급
- 최소 과금 위험을 고려해 사용량 점검

4-4. .env 파일을 왜 쓰는가

코드와 비밀정보를 분리하기 위해서다.

예를 들어 학생이 app.py를 ChatGPT에 붙여 넣고 수정 요청을 할 때, .env 파일은 보내지 않으면 된다. 그러면 API key를 노출하지 않고도 코드 수정 상담이 가능하다.

4-5. .env를 Python에서 읽는 구조

본 예시 코드에서 핵심은 아래 두 줄이다.

```
from dotenv import load_dotenv  
load_dotenv()
```

그리고

```
api_key = os.getenv("OPENAI_API_KEY")
```

즉 .env 안의 값을 읽어서 OPENAI_API_KEY라는 이름으로 불러오는 구조이다.

5. Windows Terminal에서 .py 실행하는 법과 .venv 활성화 사용법

5-1. 왜 .venv를 쓰는가

Python 표준 문서와 Python Packaging 가이드는 프로젝트별로 **가상환경(virtual environment)** 을 사용하라고 권장한다. venv는 프로젝트마다 독립된 Python 실행환경과 패키지 묶음을 만들며, 다른 프로젝트와 충돌하지 않게 해준다.

쉽게 말하면 이유는 다음 4가지이다.

1. 과목 A용 라이브러리와 과목 B용 라이브러리가 섞이지 않음
2. 내 PC 전체 Python 환경을 더럽히지 않음
3. 오류가 나면 .venv를 지우고 다시 만들기 쉬움
4. 과제별 재현성이 좋아짐

Python 공식 문서는 .venv를 프로젝트 폴더 안에 두는 방식을 예시로 들고 있으며, 활성화 후에는 그 환경 안의 python과 pip가 사용된다고 설명한다.

5-2. Windows Terminal에서 프로젝트 폴더로 이동

예를 들어 프로젝트 폴더가 바탕화면에 있다면:

```
cd %HOMEDRIVE%\Desktop\AI_Agent_Project
```

또는 D드라이브라면:

D:
`cd D:\WAI_Agent_Project`

5-3. 가상환경 만들기

Python Packaging 공식 가이드는 Windows에서 다음 명령으로 .venv를 만들라고 안내한다.

```
py -m venv .venv
```

실행하면 프로젝트 폴더 아래에 .venv 폴더가 생성된다.

5-4. 가상환경 활성화

공식 가이드의 Windows 예시는 다음과 같다.

```
.venv\Scripts\activate
```

정상 활성화되면 보통 프롬프트 앞에 (.venv)가 보인다.

예:

```
(.venv) D:\WAI_Agent_Project>
```

5-5. pip 업데이트

```
python -m pip install --upgrade pip
```

5-6. 필요한 라이브러리 설치

requirements.txt를 만들어 두었으므로 다음 한 줄이면 된다.

```
pip install -r requirements.txt
```

이 명령은 아래 패키지를 설치한다.

- openai
- streamlit
- python-dotenv
- pypdf
- requests
- beautifulsoup4

5-7. Streamlit 앱 실행

```
streamlit run app.py
```

정상 동작하면 터미널에 로컬 주소가 뜬다. 보통 브라우저가 자동으로 열리며, 열리지 않으면 표시된 주소를 직접 복사해 브라우저에 붙여 넣는다.

5-8. 종료와 재실행

앱을 멈출 때:

```
Ctrl + C
```

가상환경을 빠져나올 때:

```
deactivate
```

다음번에 다시 실행할 때는 보통 아래 4줄이면 충분하다.

```
cd D:\AI_Agent_Project  
.venv\Scripts\activate  
streamlit run app.py
```

5-9. 학생이 가장 자주 만나는 오류와 해석

오류 1. python 또는 py를 찾지 못함 원인: - Python 설치가 안 되었거나
- PATH 설정이 안 되었음

조치: - Python 재설치 - 설치 시 PATH 추가 체크

오류 2. OPENAI_API_KEY를 찾지 못했습니다 원인: - .env 파일 위치가 틀림 - 파일명이 .env.txt로 저장됨 - 변수명이 잘못됨

조치: - 프로젝트 폴더 안에 정확히 .env가 있는지 확인 - 내용이 OPENAI_API_KEY=... 형식인지 확인

오류 3. ModuleNotFoundError 원인: - 필요한 라이브러리가 설치되지 않음 - 가상환경을 활성화하지 않은 채 실행함

조치:

```
.venv\Scripts\activate  
pip install -r requirements.txt
```

오류 4. streamlit 명령을 찾지 못함 원인: - 가상환경 비활성 상태 -
Streamlit 미설치

조치:

```
.venv\Scripts\activate  
pip install streamlit
```

6. 학생 과제 수행 절차를 한 번에 요약하면

학생은 아래 순서만 따르면 된다.

1. Python 설치
 2. 프로젝트 폴더 생성
 3. app.py, .env, requirements.txt 저장
 4. Windows Terminal에서 프로젝트 폴더로 이동
 5. py -m venv .env
 6. .venv\Scripts\activate
 7. pip install -r requirements.txt
 8. streamlit run app.py
 9. 웹 화면에서 자기 과업 입력
 10. 필요하면 ChatGPT에게 app.py 수정 요청
 11. 수정된 코드를 다시 저장하고 재실행
-

7. 교수자용 수업 운영 팁

학생들에게는 처음부터 "완전한 agent"를 요구하기보다, 다음 3단계로 과제를 설계하는 것이 현실적이다.

1단계

기본 프레임 실행 성공 - .env 연결 - Streamlit 화면 실행 - 한 번의 질문/응답 성공

2단계

과업 맞춤화 - system prompt 수정 - few-shot 예시 수정 - 참고 문서 업로드 실험 - 참고 URL 결합 실험

3단계

agentic 확장 - 함수 추가 - 계산/분류/요약 자동화 - 파일 처리 자동화 - 필요 시 function calling 구조 추가

이렇게 하면 학생이 Python 문법을 깊이 몰라도, **프레임을 읽고 일부 영역만 바꾸는 방식**으로 충분히 자기 과업형 agent를 제출할 수 있다.

8. 마무리

이 가이드는 **비전공자 또는 초보자 학생**이 Windows 환경에서 OpenAI API를 사용하여 자기 과업형 agentic AI를 실습하는 데 필요한 최소 구조를 제공하는 것을 목표로 한다.

핵심은 다음 셋이다.

- 코드 전체를 처음부터 스스로 작성하려 하지 말 것
- 기본 프레임을 먼저 실행해볼 것
- 이후 ChatGPT를 이용해 자신의 과업에 맞게 반복 수정할 것

이 정도 수준만 확보해도 학생은 “프롬프트 사용자”를 넘어, **자기 과업을 수행하는 간단한 AI 도구를 직접 조립하는 사용자**로 넘어갈 수 있다.

참고 근거

- OpenAI Developer Quickstart: API key 생성 및 환경변수 사용 안내.

- OpenAI API Reference Overview: API key는 비밀번호이며 환경변수로 안전하게 로드할 것을 권장.
- OpenAI Responses API 가이드: 새 프로젝트에는 Responses API 권장, Chat Completions도 지원 지속.
- OpenAI Help Center: API key 생성 위치와 권한 설정 방식 안내.
- Python.org: Windows용 최신 Python 다운로드 제공.
- Python Packaging User Guide / Python docs: venv로 프로젝트별 가상환경 생성 및 활성화 권장.